Linux网络--Http安全部署与排障

- 1. HTTP、HTTPS 与传输安全
  - 1.1 HTTPS 提供什么
  - 1.2 TLS 握手轮廓
  - 1.3 端口与反向代理
  - 1.4 抓包工具
- 2. Web 应用安全
  - 2.1 输入验证
  - 2.2 注入与文件安全
  - 2.3 密码与认证
  - 2.4 Cookie 与会话
- 3. HTTP 服务端实现
  - 3.1 推荐模块
  - 3.2 请求读取
  - 3.3 响应发送
  - 3.4 网络地址初始化
  - 3.5 常见代码错误
- 4. Linux 进程、作业与会话
  - 4.1 进程组与作业
  - 4.2 前台与后台
  - 4.3 TTY 与会话
  - 4.4 SSH 登录
  - 4.5 服务部署
- 5. 故障排查
  - 5.1 首页正常但资源缺失
  - 5.2 页面一直等待
  - 5.3 表单参数为空
  - 5.4 登录状态异常
  - 5.5 路由匹配失败
  - 5.6 状态码与权限异常
  - 5.7 HTTPS 与 CORS 异常
- 6. 常用验证命令
- 7. 总结

1. HTTP、HTTPS 与传输安全

普通 HTTP 在传输过程中是明文的。路径、查询参数、Header 和正文可能被中间设备读取或篡改。

- GET 参数容易出现在浏览器历史、服务器日志和代理记录中。
- POST 只是把参数放入正文，不改变明文传输的事实。
- 登录、Cookie、Token 和业务数据都应通过 HTTPS 传输。

前置阅读：Linux网络--Http请求与响应、Linux网络--HttpCookie与Session

1.1 HTTPS 提供什么

HTTPS 是 HTTP 与 TLS 的组合。TLS 提供：

- 机密性：第三方无法直接读取 HTTP 内容。
- 完整性：数据被修改时可以被发现。
- 服务器身份认证：客户端通过证书确认连接对象。

HTTPS 不能自动证明业务用户是谁，也不能替代登录、权限和输入校验。

1.2 TLS 握手轮廓

1. 客户端和服务器协商 TLS 版本与密码套件。
2. 服务器发送证书链。
3. 客户端验证证书是否由受信任 CA 签发、域名是否匹配、时间是否有效。
4. 双方协商会话密钥。
5. 后续 HTTP 数据使用对称加密和完整性保护传输。

TLS 同时使用非对称机制完成身份验证和密钥协商，再使用效率更高的对称加密传输大量数据。

1.3 端口与反向代理

HTTP 习惯使用 80 端口，HTTPS 习惯使用 443 端口，但都可以配置其他端口。

常见部署是：

```
客户端 --HTTPS--> 反向代理 --HTTP--> 后端服务
```

反向代理负责 TLS、证书和公网入口，后端只处理业务。此时应正确传递原始协议、Host 和客户端地址等代理 Header，并保证内网边界可信。

1.4 抓包工具

开发时可以使用：

- 浏览器开发者工具查看请求、响应、Timing 和缓存。
- curl 获取原始响应。
- telnet 或 netcat 手工发送 HTTP。
- HTTP Toolkit 或 Fiddler 分析客户端请求。
- Wireshark 或 tcpdump 分析网络报文。

检查普通 HTTP 可以直接看到正文。分析 HTTPS 通常需要客户端信任测试 CA，或者查看服务端解密后的日志。只能在获得授权的系统和流量上抓包。

2. Web 应用安全

2.1 输入验证

服务器必须独立验证表单、查询参数、JSON、Header、Cookie 和文件上传：

- 检查类型、长度、格式和范围。
- 检查当前用户是否有权限操作目标资源。
- 对错误输入返回明确的 4xx。
- 不能只依赖前端校验。

2.2 注入与文件安全

风险	防护
SQL 注入	参数化查询或预编译语句
路径穿越	规范化路径并限制在 Web 根目录
命令注入	避免拼接 Shell 命令，限制可执行程序
XSS	按 HTML、属性、JavaScript、URL 上下文编码
CSRF	SameSite、CSRF Token、Origin/Referer 校验

上传文件时还应限制大小、类型、文件名、存储位置和执行权限，不能把用户上传目录直接作为可执行代码目录。

2.3 密码与认证

- 密码使用 Argon2id、scrypt 或 bcrypt 等合适的密码哈希算法。
- 数据库不保存明文密码。
- 登录失败信息不应泄露账号是否存在。
- 登录接口应限流，防止暴力尝试。
- HTTP Basic Authentication 中的 Base64 只是编码，不是加密。

2.4 Cookie 与会话

```
Set-Cookie: session_id=random-token; Path=/; HttpOnly; Secure; SameSite=Lax
```

Session ID 使用密码学安全随机数，登录后轮换，注销和发现风险时由服务端撤销。

`HttpOnly` 不能阻止 XSS 本身，`SameSite` 不能完全替代 CSRF Token，CORS 也不是鉴权机制。

3. HTTP 服务端实现

3.1 推荐模块

```
TCP Server
  -> 连接管理
  -> Request Reader
  -> Request Parser
  -> Router
  -> Static Handler / Dynamic Service
  -> Response Builder
  -> Response Writer
  -> 连接复用或关闭
```

3.2 请求读取

```
读取到 Header 结束空行
  -> 解析 Content-Length 或 Transfer-Encoding
  -> 循环读取完整正文
  -> 反序列化方法、URI、Header、正文
```

不能只调用一次 `read()`。TCP 是字节流，一次读取可能只得到部分 Header，也可能得到多个请求。

3.3 响应发送

1. 构造状态行。
2. 写入 Header。
3. 写入空行。
4. 写入正文。
5. 循环发送剩余字节。
6. 根据 Connection 继续读取或关闭。

3.4 网络地址初始化

```
struct sockaddr_in addr{};
addr.sin_family = AF_INET;
addr.sin_port = htons(port);
addr.sin_addr.s_addr = htonl(INADDR_ANY);
```

调用 `accept()` 时必须给长度变量正确赋值：

```
socklen_t len = sizeof(client_addr);
int client_fd = accept(server_fd,
                       reinterpret_cast<sockaddr*>(&client_addr),
                       &len);
```

3.5 常见代码错误

错误	后果
地址结构未初始化	`bind` 或 `accept` 行为异常
长度变量未初始化	`accept` 写入越界或失败

错误	后果
只读一次	请求截断或正文缺失
只写一次	响应被截断
不关闭 Socket	文件描述符耗尽
文本读取图片	二进制内容截断
路径未规范化	路径穿越
长度规则混乱	浏览器等待或连接复用失败
没有超时与大小限制	慢连接或大请求耗尽资源

4. Linux 进程、作业与会话

4.1 进程组与作业

Shell 中一次启动的命令集合可以从两个角度描述：

- 进程视角：它属于一个进程组。
- 用户视角：它是一项作业。

```
sleep 100 &
jobs
```

`&` 启动后台作业，`jobs` 显示当前 Shell 管理的作业。

4.2 前台与后台

```
fg %1
bg %1
```

- `fg` 把后台作业放到前台。
- `bg` 让暂停的作业在后台继续运行。
- `Ctrl+C` 向前台进程组发送中断信号。
- `Ctrl+Z` 暂停前台作业，把它转为暂停状态。

同一时刻通常只有一个前台作业能够接收终端输入。

4.3 TTY 与会话

TTY 表示终端设备。同一终端下启动的进程、进程组和作业通常属于同一个 Linux 会话，并具有相同 Session ID。

Linux Session ID 描述操作系统中的登录和进程归属，与 HTTP Session ID 没有关系。

4.4 SSH 登录

SSH 客户端连接远端 sshd，完成认证后启动用户 Shell，并创建伪终端。HTTP 登录则由 Web 应用验证账号密码并创建 Web Session 或 Token。

两者都属于用户认证，但协议、状态存储和生命周期不同。

4.5 服务部署

Web 服务应由 systemd、容器编排或其他进程管理器长期管理：

- 自动启动和失败重启。
- 收集日志。
- 注入配置和环境变量。
- 限制权限。
- 监听端口。
- 优雅关闭并处理已有连接。

不应依赖一个随时可能关闭的交互式终端来维持生产服务。

5. 故障排查

5.1 首页正常但资源缺失

按顺序检查：

1. HTML 中的资源 URL 是否正确。
2. 浏览器是否发出了资源请求。
3. 服务器是否找到文件。
4. 文件是否按二进制完整读取。
5. `Content-Type` 是否正确。

6. `Content-Length` 是否是真实字节数。
7. 长连接和响应边界是否一致。

5.2 页面一直等待

重点检查：

- Header 是否以空行正确结束。
- 请求正文是否读取完整。
- 响应是否缺少长度且没有合法结束方式。
- 服务端是否在发送一半时关闭连接。
- 是否错误复用了已关闭的 Socket。

5.3 表单参数为空

重点检查：

- 表单控件是否有 `name`。
- GET 参数是否位于查询字符串。
- POST 参数是否位于正文。
- 服务器是否读取了 Content-Length 指定的完整正文。
- `Content-Type` 是否与解析器一致。
- URL 编码是否被正确解码。

5.4 登录状态异常

重点检查：

- 响应是否包含 `Set-Cookie`。
- Cookie 的 Domain 和 Path 是否匹配。
- HTTPS 场景下 Cookie 是否误设或漏设 Secure。
- SameSite 是否阻止了预期请求。
- 服务端是否真正保存了 Session。
- Session 是否因超时或轮换失效。
- 注销是否同时撤销了服务端状态。

5.5 路由匹配失败

GET 的 `/Login?name=...` 不能直接匹配 `/Login`。应先拆分查询字符串，再根据方法和规范化路径查找处理函数。

5.6 状态码与权限异常

- 404：资源或路由不存在。
- 401：未认证或认证失败。
- 403：已识别身份但没有权限。
- 405：路径存在但请求方法不允许。
- 409：资源状态冲突。
- 429：请求过多。
- 500：服务端未处理异常。
- 502：网关从上游获得无效响应。
- 503：服务不可用或过载。
- 504：网关等待上游超时。

5.7 HTTPS 与 CORS 异常

HTTPS 检查证书链、域名、有效期和客户端信任。CORS 检查 OPTIONS 预检、Origin、允许的方法和 Header。

CORS 错误是浏览器安全策略的结果，不能通过设置 `Access-Control-Allow-Origin: *` 解决业务鉴权问题。

6. 常用验证命令

```
curl -i http://127.0.0.1:8080/
curl -I http://127.0.0.1:8080/
curl -v http://127.0.0.1:8080/
curl -i -X OPTIONS http://127.0.0.1:8080/
curl -i -X POST http://127.0.0.1:8080/login \
  -H "Content-Type: application/x-www-form-urlencoded" \
  --data "username=alice&password=test"
```

还可以手工发送：

```
GET / HTTP/1.1
Host: 127.0.0.1:8080
```

通过原始响应核对：

- 状态行。
- Header 分隔与空行。
- Content-Type。
- Content-Length。
- 正文是否完整。
- Connection 语义是否与服务端行为一致。

7. 总结

HTTP 系列的完整链路是：

```
URL
  -> DNS
  -> IP:端口
  -> TCP 或 QUIC
  -> 请求报文
  -> 请求解析和正文分帧
  -> 静态文件或动态路由
  -> 响应报文
  -> 浏览器解析并继续加载资源
  -> Cookie 或 Token 保存状态
  -> HTTPS 保护传输
  -> 部署、监控与故障排查
```

HTTP 难点通常不在单个 Header 名称，而在这些环节之间的边界：连接是否复用、正文是否完整、路径是否安全、状态由谁保存、缓存是否命中，以及客户端和服务器是否对同一报文作出一致解释。

返回系列第一篇：Linux网络--Http请求与响应